DevPulse API

Security Architecture & Security Standards

Version: 1.0
Status: Approved
Document Type: Security Architecture
Compliance: Mandatory
Classification: Internal Engineering Document

Related Documents - 02-software-architecture.md - 05-api-specification.md - 06-api-standards.md - 07-coding-standards.md

---

1. Purpose

تحدد هذه الوثيقة الاستراتيجية الأمنية الرسمية لمشروع **DevPulse API**.

هدفها هو توحيد جميع القرارات الأمنية المتعلقة بالنظام، وتقليل المخاطر الأمنية أثناء التطوير، وضمان الالتزام بأفضل الممارسات.

---

2. Security Principles

يعتمد المشروع المبادئ التالية:

- Security by Design
- Secure by Default
- Least Privilege
- Defense in Depth
- Fail Securely
- Zero Trust for User Input
- Principle of Least Knowledge

---

3. Security Objectives

يجب أن يحقق النظام:

- Confidentiality
- Integrity
- Availability
- Accountability
- Auditability

---

4. Authentication Strategy

Selected Method

Laravel Sanctum

Authentication Flow

```
User

↓

Login Request

↓

Credential Validation

↓

Generate Personal Access Token

↓

Authenticated Requests

↓

Logout

↓

Token Revoked
```

Rules

MUST

- استخدام HTTPS في الإنتاج.
- استخدام Bearer Token.
- إلغاء التوكن عند تسجيل الخروج.
- تخزين كلمات المرور باستخدام Hash.

---

5. Authorization Strategy

يعتمد المشروع على:

Laravel Policies

كل Resource يمتلك Policy مستقلة.

Examples

- PostPolicy
- CommentPolicy
- CategoryPolicy

Ownership Rules

User

يمكنه إدارة موارده فقط.

Admin

يمكنه إدارة جميع الموارد.

---

6. Access Control Matrix

Resource	Guest	User	Admin
Posts	Read	CRUD Own	CRUD All
Categories	Read	Read	CRUD
Tags	Read	Read	CRUD
Comments	Read	CRUD Own	CRUD All
Users	Register/Login	Own Profile	Manage All

---

7. Threat Model

التهديدات الأساسية:

Authentication Bypass

Mitigation

- Sanctum
- Policies

Broken Authorization

Mitigation

- Ownership Checks
- Policies

SQL Injection

Mitigation

- Eloquent ORM
- Parameter Binding
- Validation

XSS

Mitigation

- Output Escaping في العملاء.
- منع إدخال البيانات الضارة عند الحاجة.

Mass Assignment

Mitigation

استخدام:

```
$fillable
```

أو

```
$guarded
```

بشكل صحيح.

File Upload Abuse

Mitigation

- MIME Validation
- Size Validation
- Random File Names
- Storage Isolation

Brute Force

Mitigation

- Rate Limiting
- Login Throttling

---

8. Input Validation Strategy

جميع البيانات القادمة من المستخدم

MUST

- تمر عبر Form Requests.
- يتم التحقق من أنواع البيانات.
- يتم رفض أي بيانات غير معروفة.
- يتم التحقق من العلاقات الموجودة في قاعدة البيانات.

---

9. Password Policy

Passwords MUST

- تكون مشفرة باستخدام Hash.
- لا يتم تسجيلها في Logs.
- لا يتم إرسالها في Responses.
- لا يتم حفظها كنص صريح.

---

10. API Security

كل Endpoint

MUST

- يحدد هل يحتاج Authentication.

- يحدد هل يحتاج Authorization.
- يستخدم Validation.
- يعيد أخطاء بصيغة موحدة.

---

11. File Upload Security

قبل حفظ أي ملف:

MUST

- التحقق من نوع الملف.
- التحقق من الحجم.
- إنشاء اسم فريد.
- منع تنفيذ الملفات المرفوعة.
- تخزين الملفات في المكان المناسب وفق سياسة المشروع.

---

12. Database Security

Rules

- استخدام Foreign Keys.
- استخدام Unique Constraints.
- منع البيانات المكررة.
- منع العلاقات غير الصحيحة.

---

13. Sensitive Data

يحظر إرجاع:

- password
- remember_token
- access tokens
- internal identifiers غير الضرورية
- أي بيانات سرية لا يحتاجها العميل

---

14. Logging Strategy

يسجل النظام:

- Authentication Failures
- Unexpected Exceptions
- Critical Errors

لا يسجل:

- Passwords
- Tokens
- Secrets
- Personal Sensitive Information

---

15. Environment Security

يجب أن تكون:

- APP_KEY محفوظة.
- مفاتيح الخدمات داخل البيئة فقط.
- عدم رفع ملفات البيئة إلى Git.

---

16. Error Handling Security

يحظر:

- إظهار Stack Trace.
- كشف SQL Queries.
- كشف أسماء الملفات الداخلية.
- كشف تفاصيل Exceptions في الإنتاج.

---

17. Rate Limiting

Public Endpoints

MUST

- تكون محددة بمعدل مناسب.

Authentication Endpoints

MUST

- تمتلك حماية إضافية ضد Brute Force.

---

18. Security Headers

في بيئة الإنتاج يومى باستخدام:

- X-Content-Type-Options
- X-Frame-Options
- Referrer-Policy
- Content-Security-Policy (عند الحاجة)
- Strict-Transport-Security

---

19. Security Checklist

قبل اعتماد أي Feature:

- هل يوجد Validation؟
- هل يوجد Policy؟
- هل توجد حماية من Mass Assignment؟
- هل البيانات الحساسة مخفية؟
- هل يوجد Rate Limiting عند الحاجة؟
- هل يوجد Feature Test للحالات الأمنية؟
- هل تمت مراجعة الأخطاء؟

---

20. Incident Response Guidelines

عند اكتشاف ثغرة:

1. توثيق المشكلة.
2. تقييم التأثير.
3. إصلاح السبب الجذري.
4. إضافة اختبار يمنع تكرارها.
5. تحديث Architecture Decisions إذا لزم الأمر.

---

21. Security Review Checklist

يجب أن يجيب على كل Pull Request:

- هل توجد أي بيانات حساسة في Response؟
- هل تمت مراجعة الصلاحيات؟
- هل يمكن للمستخدم الوصول إلى بيانات غيره؟
- هل توجد مدخلات غير محمية؟
- هل الكود يلتزم بالوثيقة الأمنية؟

---

22. Future Security Enhancements

قد يضاف مستقبلاً:

- Two-Factor Authentication
- Email Verification
- Audit Logs
- Security Events
- Device Management
- Token Expiration Policies
- Intrusion Detection
- Secret Rotation

---

23. Revision History

Version	Date	Description
1.0	2026-07-21	Initial Security Architecture